COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	October 05, 2026	**DATE SUBMITTED:**	October 05, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 10.1 Graphs

1. PROCEDURE OUTPUT

ILO A: Create C++ code for graph implementation utilizing adjacency matrix and adjacency list

A.1. Create a Graph (Code)

```cpp
#ifndef GRAPH_H
#define GRAPH_H

#include <iostream>
#include <vector>

template <typename T>
class Graph{

    private:
        size_t V;
        std::vector<std::vector<T>> adjacencyList;

    public:
        Graph(size_t N) : V(N)
        {
            adjacencyList.resize(V);
        }

        void add_edge(T src, T dest)
        {
            if (src >= 0 && src < V &&
            dest >= 0 && dest < V)
            {
                adjacencyList[src].push_back(dest);
            }
            else
            {
                std::cerr<<"Vertex out of bounds"<<std::endl;
            }
        }

        void display_vertices()
        {
            std::cout<<"Graph Vertices: ";
```

```
    for (size_t i = 0; i < V; i++)
    {
        std::cout<<i<<" ";
    }
    std::cout<<std::endl;
}

void display_graph()
{
    std::cout<<"\nGraph:"<<std::endl;

    for (size_t i=0; i < V; i++)
    {
        std::cout<<i<<" -> ";

        for(auto vertex : adjacencyList[i])
        {
            std::cout<<vertex<<" ";
        }
        std::cout<<std::endl;
    }
}
```

```cpp
#include <iostream>
#include "graph.h"

int main()
{
    using T = unsigned;

    Graph<T> G(6);
    G.add_edge(0, 1);
    G.add_edge(0, 2);
    G.add_edge(1, 4);
    G.add_edge(2, 3);
    G.add_edge(3, 1);
    G.add_edge(4, 3);

    G.display_vertices();

    G.display_graph();

    return 0;
}
```

Output

```
Graph Vertices: 0 1 2 3 4 5

Graph:
0 -> 1 2
1 -> 4
2 -> 3
3 -> 1
4 -> 3
5 ->
```

A.2. Adjacency Matrix (code)

```
#ifndef ADJACENCY_MATRIX_H
#define ADJACENCY_MATRIX_H

#include <iostream>

template <typename T>
class Graph
{
    private:
        int vertices;
        int matrix[10][10];

    public:
        Graph(int v)
        {
            vertices = v;

            for(int i = 0; i < vertices; i++)
            {
                for(int j = 0; j<vertices; j++)
                {
                    matrix[i][j] = 0;
                }
            }
        }

        void addEdge(int start, int end){
            matrix[start][end] = 1;
            matrix[end][start] = 1;
        }

        void displayMatrix()
        {
            std::cout<<"Adjacency Matrix: \n\n";

            for (int i = 0; i < vertices; i++)
            {
                for(int j = 0; j<vertices; j++)
                {
                    std::cout<<matrix[i][j]<<" ";
                }
                std::cout<<std::endl;
            }
        }
};

#endif
```

```cpp
#include <iostream>
#include "adjacency_matrix.h"

int main() {

    Graph<int> graph(5);

    graph.addEdge(0,1);
    graph.addEdge(0,2);
    graph.addEdge(1,3);
    graph.addEdge(2,4);
    graph.addEdge(3,4);

    graph.displayMatrix();

    return 0;
}
```

Output

```
Adjacency Matrix:

0 1 1 0 0
1 0 0 1 0
1 0 0 0 1
0 1 0 0 1
0 0 1 1 0
```

A.3. Adjacency List (code)

```
#ifndef ADJACENCY_LIST_H
#define ADJACENCY_LIST_H

#include <iostream>

struct adjNode{
    int val, cost;
    adjNode* next;
};

struct graphEdge{
    int start_ver, end_ver, weight;
};

class DiaGraph{
    adjNode* getAdjListNode(int value, int weight, adjNode* head){
        adjNode* newNode = new adjNode;
        newNode->val = value;
        newNode->cost = weight;
        newNode->next = head;
        return newNode;
    }

    int N;
```

```
public:
    adjNode **head;

    DiaGraph(graphEdge edges[], int n, int N){

        head = new adjNode*[N]();
        this->N = N;

        for (int i = 0; i < N; ++i){
            head[i] = nullptr;
        }

        for(unsigned i = 0; i < n; i++){
            int start_ver = edges[i].start_ver;
            int end_ver = edges[i].end_ver;
            int weight = edges[i].weight;

            adjNode* newNode = getAdjListNode(end_ver, weight, head[start_ver]);

            head[start_ver] = newNode;
        }
    }
```

```
    ~DiaGraph(){
        for (int i = 0; i < N; i++){
            delete[] head[i];
        }

        delete[] head;
    }
};

void display_AdjList(adjNode* ptr, int i){
    while (ptr != nullptr){
        std::cout<<"("<<i<<","<<ptr->val<<", "<<ptr->cost<<") ";
        ptr = ptr->next;
    }
    std::cout<<std::endl;
};

#endif
```

```
#include <iostream>
#include "adjacency_list.h"

int main()
{
    graphEdge edges[] = {
        {0,1,2},
        {0,2,4},
        {1,4,3},
        {2,3,2},
        {3,1,4},
        {4,3,3}
    };

    int N = 6;

    int n = sizeof(edges) / sizeof(edges[0]);

    DiaGraph diagraph(edges, n, N);

    std::cout<<"Graph adjacency list "<<std::endl<<"(Start vertex, end_vertex, weight): "<<std::endl;

    for (int i = 0; i < N; i++)
    {
        display_AdjList(diagraph.head[i], i);
    }

    return 0;
}
```

Output

```
Graph adjacency list
(Start vertex, end_vertex, weight):
(0,2, 4) (0,1, 2)
(1,4, 3)
(2,3, 2)
(3,1, 4)
(4,3, 3)
```

ILO B: Create C++ code for implementing graph traversal algorithms such as Breadth-First and Depth-First Search

B.1. Depth-First Search (Code)

```
#ifndef DFS_H
#define DFS_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <stack>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator<(const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }
    inline bool operator> (const Edge<T> &e) const
    {
        return this->weight >e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i<G.vertices(); i++)
    {
        os<<i<<":\t";
        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
        {
            os<<"{"<<e.dest<<": "<<e.weight<<"}, ";
        }
        os<<std::endl;
    }
    return os;
};

template <typename T>
class Graph
{
    public:
        Graph(size_t N) : V(N)
        {
        }

        auto vertices() const
        {
            return V;
        }

        auto &edges() const
        {
            return edge_list;
        }
```

```
void add_edge (Edge<T> &&e)
{
    if (e.src >= 1 && e.src <= V &&
    e.dest >= 1 && e.dest <= V)
    {
    edge_list.emplace_back(e);
    }
    else
    {
        std::cerr<<"Vertex out of bounds"<<std::endl;
    }
}

auto outgoing_edges (size_t v) const
{
    std::vector<Edge<T>> edges_from_v;
    for (auto &e : edge_list)
    {
        if(e.src == v)
        {
            edges_from_v.emplace_back(e);
        }
    }
    return edges_from_v;
}

template <typename U>
friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);
```

```
    private:
        size_t V;
        std::vector<Edge<T>> edge_list;
};

template <typename T>
auto depth_first_search (const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);
    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 0}, {5, 0}};
    edges[2] = {{1, 0}, {5, 0}, {4, 0}};
    edges[3] = {{4, 0}, {7, 0}};
    edges[4] = {{2, 0}, {3, 0}, {5, 0}, {6, 0}, {8, 0}};
    edges[5] = {{1, 0}, {2, 0}, {4, 0}, {8, 0}};
    edges[6] = {{4, 0}, {7, 0}, {8, 0}};
    edges[7] = {{3, 0}, {6, 0}};
    edges[8] = {{4, 0}, {5, 0}, {6, 0}};
    for (auto &i : edges)
    for (auto &j : i.second)
    G.add_edge(Edge<T>{i.first, j.first, j.second});
    return G;
}
```

```cpp
template <typename T>
void test_DFS()
{

    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "DFS Order of vertices: " << std::endl;

    auto dfs_visit_order = depth_first_search(G, 1);
    for (auto v : dfs_visit_order)
    {
        std::cout << v << std::endl;
    }
}



#endif
```

```cpp
#include <iostream>
#include "DFS.h"

int main()
{
    using T = unsigned;

    test_DFS<T>();

    return 0;
}
```

Output

```
1:      {2: 0}, {5: 0},
2:      {1: 0}, {5: 0}, {4: 0},
3:      {4: 0}, {7: 0},
4:      {2: 0}, {3: 0}, {5: 0}, {6: 0}, {8: 0},
5:      {1: 0}, {2: 0}, {4: 0}, {8: 0},
6:      {4: 0}, {7: 0}, {8: 0},
7:      {3: 0}, {6: 0},
8:      {4: 0}, {5: 0}, {6: 0},

DFS Order of vertices:
1
5
8
6
7
3
4
2
```

B.2. Breadth-First Search

```
#ifndef BFS_H
#define BFS_H

#include <string>
#include <vector>
#include <iostream>
#include <set>
#include <map>
#include <queue>

template <typename T>
class Graph;

template <typename T>
struct Edge
{
    size_t src;
    size_t dest;
    T weight;

    inline bool operator< (const Edge<T> &e) const
    {
        return this->weight < e.weight;
    }

    inline bool operator > (const Edge<T> &e) const
    {
        return this -> weight > e.weight;
    }
};
```

```cpp
template <typename T>
std::ostream &operator<<(std::ostream &os, const Graph<T> &G)
{
    for (auto i = 1; i < G.vertices(); i++)
    {
        os<<i<<":\t";

        auto edges = G.outgoing_edges(i);
        for (auto &e : edges)
        {
            os<<"{"<<e.dest<<": "<<e.weight<<"}, ";

            os<<std::endl;
        }
    }
};

template <typename T>
class Graph
{
    public:
        Graph(size_t N) : V(N)
        {
        }

        auto vertices() const
        {
            return V;
        }

        auto &edges() const
        {
            return edge_list;
        }

        void add_edge(Edge<T> &&e)
        {
            if (e.src >= 1 && e.src <= V &&
            e.dest >= 1 && e.dest <= V)
            {
                edge_list.emplace_back(e);
            }
            else{
```

```
            std::cerr<<"Vertex out of bounds"<<std::endl;
        }
    }

    auto outgoing_edges(size_t v) const
    {
        std::vector<Edge<T>> edges_from_v;
        for (auto &e : edge_list)
        {
            if (e.src == v)
            {
                edges_from_v.emplace_back(e);
            }
        }
        return edges_from_v;
    }

    template <typename U>
    friend std::ostream &operator<<(std::ostream &os, const Graph<U> &G);

private:
    size_t V;
    std::vector<Edge<T>> edge_list;
};

template <typename T>
auto create_reference_graph()
{
    Graph<T> G(9);

    std::map<unsigned, std::vector<std::pair<size_t, T>>> edges;
    edges[1] = {{2, 2}, {5, 3}};
    edges[2] = {{1, 2}, {5, 5}, {4, 1}};
    edges[3] = {{4, 2}, {7, 3}};
    edges[4] = {{2, 1}, {3, 2}, {5, 2}, {6, 4}, {8, 5}};
    edges[5] = {{1, 3}, {2, 5}, {4, 2}, {8, 3}};
    edges[6] = {{4, 4}, {7, 4}, {8, 1}};
    edges[7] = {{3, 3}, {6, 4}};
    edges[8] = {{4, 5}, {5, 3}, {6, 1}};
```

```
    for (auto &i : edges)
    {
        for (auto &j : i.second)
        {
            G.add_edge(Edge<T>{i.first, j.first, j.second});
        }
    }
    return G;
}

template <typename T>
auto breadth_first_search(const Graph<T> &G, size_t dest)
{
    std::queue<size_t> queue;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    queue.push(1);

    while (!queue.empty())
    {
        auto current_vertex = queue.front();
        queue.pop();
        if(visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                queue.push(e.dest);
            }
        }
    }
    return visit_order;
}
```

```cpp
template <typename T>
void test_BFS()
{

    auto G = create_reference_graph<unsigned>();
    std::cout << G << std::endl;

    std::cout << "BFS Order of vertices: " << std::endl;
    auto bfs_visit_order = breadth_first_search(G, 1);

    for (auto v : bfs_visit_order)
    {
        std::cout << v << std::endl;
    }
}



#endif
```

```cpp
#include <iostream>
#include "BFS.h"

int main()
{
    using T = unsigned;
    test_BFS<T>();
    return 0;
}
```

Output

```
1:      {2: 2},
{5: 3},
2:      {1: 2},
{5: 5},
{4: 1},
3:      {4: 2},
{7: 3},
4:      {2: 1},
{3: 2},
{5: 2},
{6: 4},
{8: 5},
5:      {1: 3},
{2: 5},
{4: 2},
{8: 3},
6:      {4: 4},
{7: 4},
{8: 1},
7:      {3: 3},
{6: 4},
8:      {4: 5},
{5: 3},
{6: 1},
```

Observation
For ILO A, the code works to create a graph by using an adjacency matrix and adjacency list. The adjacency matrix uses a two-dimensional array where values show whether vertices are connected and adjacency list uses nodes to store the connected vertices and the edge weights. For ILO B, the DFS and BFS work by visiting the vertices of a graph in different orders. For DFS, it uses a stack-based approach to go through a path before backtracking. BFS uses a queue to visit nearby vertices first. In the output, we are able to see how the graph connections are stored and the traversal algorithms to determine the order of the vertices that are visited.

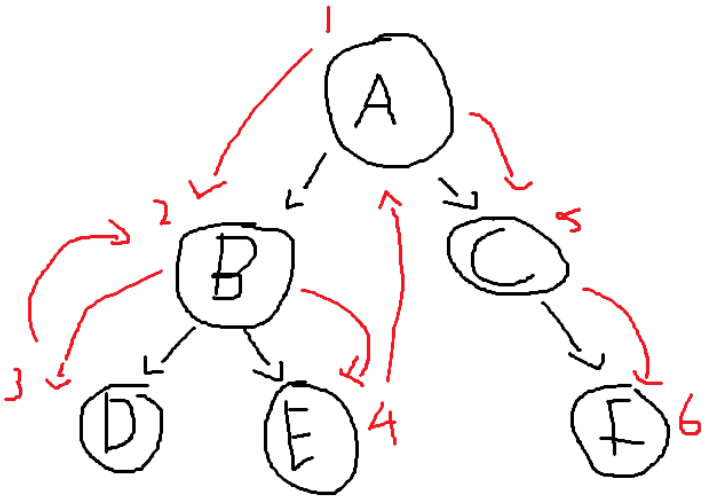
2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Demonstrate an understanding of graph implementation, operations and traversal methods.

1. A person wants to visit different locations indicated on a map. He starts from one location (vertex) and wants to visit every vertex until it finishes from one vertex, backtracks, and then explore other vertex from same vertex. Discuss which algorithm would be most helpful to accomplish this task.
    - The best algorithm to use for this situation is the Depth-First Search or DFS since it starts from a chosen vertex and then explores one connected path as deep as possible then backtracks to the previous vertex. Once a path is finished, it returns to the previous vertex and then checks for an unvisited vertex and explores it. The process continues until all vertices have been visited which matches the person's goal of finishing a route, backtracking, and then exploring another route.

2. Identify the equivalent of DFS in traversal strategies for trees. To efficiently answer this question, provide a graphical comparison, examine pseudocode and code implementation.

GRAPH:



DFS: A->B->D->E->C->F
Tree (Preorder): A->B->D->E->C->F

Pseudo code:

```
DFS:

DFS(vertex):
    mark vertex as visited

    for each adjacent vertex:
        if it is not visited:
            DFS(adjacent vertex)

PREORDER:

Preorder(node):
    if node is not NULL:
        visit node
        Preorder(node.left)
        Preorder(node.right)
```

Code implementation:

DFS

```cpp
template <typename T>
auto depth_first_search (const Graph<T> &G, size_t dest)
{
    std::stack<size_t> stack;
    std::vector<size_t> visit_order;
    std::set<size_t> visited;
    stack.push(1);

    while (!stack.empty())
    {
        auto current_vertex = stack.top();
        stack.pop();
        if (visited.find(current_vertex) == visited.end())
        {
            visited.insert(current_vertex);
            visit_order.push_back(current_vertex);
            for (auto e : G.outgoing_edges(current_vertex))
            {
                if(visited.find(e.dest) == visited.end())
                {
                    stack.push(e.dest);
                }
            }
        }
    }
    return visit_order;
}
```

Preorder

```
template <typename T>
void GeneralTree<T>::preOrder(){

    std::cout << key << " ";

    if(firstChild != nullptr){
        firstChild->preOrder();
    }

    if(sibling != nullptr){
        sibling->preOrder();
    }
}
```

3. In the performed code, what data structure is used to implement the Breadth First Search?
   - In the performed code, the queue data structure is used to implement BFS. It follows a first in, first out principle to allow the algorithm to visit vertices level by level.

4. How many times can a node be visited in the BFS?
   - In BFS, a node is visited only once. Once a node is visited, the algorithm does not visit it again. This prevents the algorithm from repeating the process and allows BFS to efficiently traverse a graph level by level.

Conclusion

In this activity, I learned how graphs work. I learned how they can be represented using adjacency matrices and adjacency lists. Graphs can be traversed using DFS and BFS in different ways. DFS explores a path as deep as possible before backtracking, while BFS uses a queue to visit vertices level by level. The activity also helped me to understand how stacks and queues can be applied. I believe that I was not able to master graphs fully and I am hoping to learn more about it. Overall, I was able to gain a better understanding of graph implementations, traversal algorithms, and their outputs.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.